# Uncertainty Estimation

Comprehensive Cheatsheet

Machine Intelligence - Andrés Masegosa - Fall 2025

## Contents


**1 Why Uncertainty Matters** **2**

1.1 The Problem with Overconfident Models . . . 2

1.2 Types of Uncertainty . . . 2

**2 Neural Networks as Probabilistic Models** **3**

2.1 The Softmax Function . . . 3

2.2 Building a Probabilistic CNN . . . 3

**3 Measuring Uncertainty with Entropy** **5**

3.1 Information Entropy . . . 5

3.2 Computing Entropy in Python . . . 6

3.3 Entropy Reveals Model Behavior . . . 7

**4 Selective Prediction** **8**

4.1 The Idea: Abstaining When Uncertain . . . 8

4.2 Coverage and Accuracy . . . 9

4.3 Implementation in Python . . . 9

**5 Ensemble Methods for Better Uncertainty** **11**

5.1 Why Ensembles? . . . 11

5.2 Building an Ensemble . . . 11

5.3 Implementation in Python . . . 11

5.4 Combining Ensemble Predictions . . . 12

5.5 Entropy for Ensembles . . . 13

**6 Out-of-Distribution Detection** **14**

6.1 The OOD Problem . . . 14

6.2 Using Uncertainty for OOD Detection . . . 14

**7 Complete Implementation Example** **15**

7.1 Full Pipeline: Training to Uncertainty-Aware Prediction . . . . . 15

**8 Key Takeaways** **17**

# 1 Why Uncertainty Matters

## 1.1 The Problem with Overconfident Models

**The Need for Uncertainty**

**Key Question**: When a model makes a prediction, how confident should we be?
**The Problem**:

- Traditional ML models output a single prediction
- They don't tell you *how sure* they are
- A model predicting "cat" with 51% confidence looks the same as 99% confidence
- This is dangerous in critical applications!

**Real-World Consequences**:

- **Medical diagnosis**: Model says "cancer" but is only 52% sure
- **Autonomous driving**: Model says "pedestrian" but data is corrupted
- **Financial decisions**: Model recommends action on uncertain prediction

**Solution**: Probabilistic models that output **distributions** instead of point predictions!

## 1.2 Types of Uncertainty

**Two Types of Uncertainty**

1. **Aleatoric Uncertainty** (Data Uncertainty):
   - Inherent randomness in the data
   - Cannot be reduced by collecting more data
   - Example: Handwritten digit that looks like both 4 and 9
   - Example: Noisy sensor measurements
2. **Epistemic Uncertainty** (Model Uncertainty):
   - Uncertainty due to lack of knowledge
   - CAN be reduced by collecting more data
   - Example: Model hasn't seen examples from a certain region
   - Example: Out-of-distribution data (corrupted images)

**Plain English**:

- **Aleatoric**: "The data itself is ambiguous"
- **Epistemic**: "I haven't learned enough to be confident"

# 2 Neural Networks as Probabilistic Models

## 2.1 The Softmax Function

**Softmax: Converting Scores to Probabilities**

**Purpose**: Convert raw neural network outputs (logits) into a probability distribution over classes.
**Formula**: For a vector $\vec{z} = [z_1, z_2, \ldots, z_K]$ of $K$ logits:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

**Properties**:

1. **Normalization**: All outputs sum to 1:

$$\sum_{i=1}^{K} \text{softmax}(z_i) = 1$$

2. **Non-negative**: All outputs are in $(0, 1)$
3. **Amplification**: Larger logits get exponentially more probability
4. **Differentiable**: Works with gradient descent

**Interpretation**: Output $i$ represents $P(\text{class} = i \mid \vec{x})$

**Softmax Calculation Example**

**Given logits**: $\vec{z} = [2.0, 1.0, 0.1]$ (3 classes)
**Step 1**: Compute exponentials:

$$e^{2.0} \approx 7.39$$
$$e^{1.0} \approx 2.72$$
$$e^{0.1} \approx 1.11$$

**Step 2**: Compute sum:

$$7.39 + 2.72 + 1.11 = 11.22$$

**Step 3**: Compute probabilities:

$$P(\text{class } 0) = \frac{7.39}{11.22} \approx 0.658 \quad (65.8\%)$$
$$P(\text{class } 1) = \frac{2.72}{11.22} \approx 0.242 \quad (24.2\%)$$
$$P(\text{class } 2) = \frac{1.11}{11.22} \approx 0.099 \quad (9.9\%)$$

**Interpretation**: Model is 65.8% confident in class 0, somewhat uncertain between classes 0 and 1.

## 2.2 Building a Probabilistic CNN

```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

```python
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.optimizers import RMSprop

# Define CNN with softmax output (probabilistic model)
cnn_model = Sequential([
    Conv2D(kernel_size=(5, 5), filters=8,
           activation='relu', padding='VALID',
           input_shape=(28, 28, 1)),
    MaxPooling2D(pool_size=(6, 6)),
    Flatten(),
    # Softmax makes output a probability distribution!
    Dense(units=10, activation='softmax')
])

cnn_model.compile(
    loss=SparseCategoricalCrossentropy(),
    optimizer=RMSprop(),
    metrics=['accuracy']
)

# Train the model
cnn_model.fit(x_train, y_train, epochs=5)

# Get probabilities (not just predictions!)
probabilities = cnn_model.predict(x_test)
# Shape: (num_samples, 10) - probability for each class
```

**Key Insight: Softmax Gives Probabilities**

**Without softmax**: Model outputs raw scores (logits) with no meaning

$$\text{logits} = [2.5, -1.3, 0.8, \ldots]$$

**With softmax**: Model outputs valid probability distribution

$$P(\text{class} \mid \vec{x}) = [0.65, 0.05, 0.15, \ldots]$$

**This allows us to**:

- See how confident the model is
- Identify uncertain predictions
- Make selective predictions (abstain when unsure)

# 3 Measuring Uncertainty with Entropy

## 3.1 Information Entropy

**Entropy: Measuring Uncertainty**

**Definition**: Entropy measures the "expected surprise" or uncertainty of a probability distribution.
**Formula**: For predicted probabilities $\vec{p} = [p_1, p_2, \ldots, p_K]$:

$$H = -\sum_{j=1}^{K} p_j \log_2(p_j)$$

**Properties**:

- **Range**: $0 \leq H \leq \log_2(K)$
- **Minimum** ($H = 0$): Model is 100% confident (one class has probability 1)
- **Maximum** ($H = \log_2(K)$): Uniform distribution (equal probability for all classes)

**Units**:

- Using $\log_2$: Entropy in **bits**
- Using ln: Entropy in **nats**

**Plain English**: High entropy = model is unsure. Low entropy = model is confident.

**Interpreting Entropy Values**

**For 10-class classification** (like MNIST digits):

- Maximum entropy: $\log_2(10) \approx 3.32$ bits
- This means: "I'm equally unsure about all 10 classes"

**Useful interpretation**: $2^H$ gives the "effective number of classes"

- $H = 0$ bits $\rightarrow 2^0 = 1$ effective class (certain)
- $H = 1$ bit $\rightarrow 2^1 = 2$ effective classes (unsure between 2)
- $H = 2$ bits $\rightarrow 2^2 = 4$ effective classes
- $H = 3.32$ bits $\rightarrow 2^{3.32} = 10$ effective classes (maximum uncertainty)

**Example**:

- Model predicts $[0.9, 0.1, 0, 0, \ldots]$: Low entropy (confident)
- Model predicts $[0.3, 0.3, 0.2, 0.1, 0.1]$: High entropy (uncertain)

**Entropy Calculation Examples**

**Example 1: Confident Prediction**
Probabilities: $[0.95, 0.03, 0.02, 0, 0, 0, 0, 0, 0, 0]$

$$\begin{aligned} H &= -(0.95 \log_2 0.95 + 0.03 \log_2 0.03 + 0.02 \log_2 0.02) \\ &= -(0.95 \times (-0.074) + 0.03 \times (-5.06) + 0.02 \times (-5.64)) \\ &\approx 0.07 + 0.15 + 0.11 \\ &\approx 0.33 \text{ bits} \end{aligned}$$

$2^{0.33} \approx 1.26$ effective classes. Model is very confident!
**Example 2: Uncertain Prediction**
Probabilities: $[0.25, 0.25, 0.20, 0.15, 0.10, 0.05, 0, 0, 0, 0]$

$$H \approx 2.16 \text{ bits}$$

$2^{2.16} \approx 4.5$ effective classes. Model is quite uncertain!

## 3.2 Computing Entropy in Python

```python
import numpy as np

def compute_entropy(probabilities):
    """
    Compute entropy of probability distributions.

    Args:
        probabilities: (batch_size, num_classes) array
    Returns:
        entropy: (batch_size,) array of entropy values in bits
    """
    # Add small epsilon to avoid log(0)
    probs = np.clip(probabilities, 1e-10, 1.0)
    entropy = -np.sum(probs * np.log2(probs), axis=1)
    return entropy

# Get model predictions
probs = cnn_model.predict(x_test)

# Compute entropy for each prediction
entropies = compute_entropy(probs)

# Find most uncertain predictions
most_uncertain_idx = np.argsort(entropies)[-10:]  # Top 10 uncertain
print(f"Most uncertain predictions: {most_uncertain_idx}")
print(f"Their entropies: {entropies[most_uncertain_idx]}")

# Effective number of classes
effective_classes = 2 ** entropies
print(f"Effective classes: {effective_classes[most_uncertain_idx]}")
```

### 3.3 Entropy Reveals Model Behavior

**Key Observation: Models Know When They're Wrong**

**Empirical Finding**: Models tend to have higher entropy on predictions they get wrong!
**What this means**:

- When model is correct: Usually low entropy (confident)
- When model is wrong: Usually higher entropy (uncertain)

**Why this is valuable**:

1. Model "knows" when it might be wrong
2. We can use entropy to filter unreliable predictions
3. Enables selective prediction (abstain when uncertain)

**Visualizing Entropy Distribution**:

```
# Separate correct and incorrect predictions
predictions = np.argmax(probs, axis=1)
correct_mask = predictions == y_test

# Entropy for correct vs incorrect
entropy_correct = entropies[correct_mask]
entropy_incorrect = entropies[~correct_mask]

print(f"Mean entropy (correct): {np.mean(entropy_correct):.3f}")
print(f"Mean entropy (incorrect): {np.mean(entropy_incorrect):.3f}")
# Typically: incorrect predictions have higher entropy!
```

# 4 Selective Prediction

## 4.1 The Idea: Abstaining When Uncertain

**Selective Prediction**

**Core Idea**: Don't make predictions when the model is too uncertain!
**How it works**:

1. Set a **confidence threshold** $\tau$
2. For each prediction, check if $\max_i P(\text{class}_i) > \tau$
3. If yes: Make the prediction
4. If no: Abstain ("I'm not sure")

**Trade-off**:

- Higher threshold $\tau \rightarrow$ Fewer predictions, higher accuracy
- Lower threshold $\tau \rightarrow$ More predictions, lower accuracy

**Use Cases**:

- Medical diagnosis: Refer uncertain cases to human doctors
- Autonomous driving: Hand control to human driver when unsure
- Content moderation: Flag uncertain content for human review

## 4.2 Coverage and Accuracy

**Key Metrics for Selective Prediction**

**Coverage**: Fraction of instances where model makes a prediction

$$\text{Coverage} = \frac{\text{Number of predictions made}}{\text{Total number of instances}}$$

**Selective Accuracy**: Accuracy over instances where predictions are made

$$\text{Selective Accuracy} = \frac{\text{Correct predictions made}}{\text{Total predictions made}}$$

**Example**:

- 1000 test samples
- Threshold $\tau = 0.9$ (require 90% confidence)
- Model makes 800 predictions, abstains on 200
- Of 800 predictions, 780 are correct

$$\text{Coverage} = \frac{800}{1000} = 80\%$$
$$\text{Selective Accuracy} = \frac{780}{800} = 97.5\%$$

**Compare to**: Regular accuracy (without selection) might be 95%
We sacrifice 20% coverage to gain 2.5% accuracy improvement!

## 4.3 Implementation in Python

```python
def accuracy_by_threshold(model, x, labels, threshold):
    """
    Compute accuracy and coverage for a given confidence threshold.

    Returns: (threshold, accuracy_percent, coverage_percent)
    """
    probs = model.predict(x)

    # Find maximum probability for each sample
    max_probs = np.max(probs, axis=1)

    # Which samples exceed threshold?
    above_threshold = max_probs > threshold

    # Which predictions are correct?
    predictions = np.argmax(probs, axis=1)
    correct = predictions == np.squeeze(labels)

    # Compute metrics
    num_above = np.sum(above_threshold)
    if num_above == 0:
        return threshold, 0.0, 0.0

    # Accuracy among those above threshold
    accuracy = 100 * np.sum(above_threshold & correct) / num_above

```

```python
    # Coverage (fraction of samples above threshold)
    coverage = 100 * num_above / len(labels)

    return threshold, accuracy, coverage

# Test different thresholds
thresholds = np.arange(0.0, 1.0, 0.05)
results = [accuracy_by_threshold(cnn_model, x_test, y_test, t)
           for t in thresholds]

# Find threshold for 99% accuracy
for thresh, acc, cov in results:
    if acc >= 99.0:
        print(f"Threshold {thresh:.2f}: {acc:.1f}% accuracy, {cov:.1f}% coverage")
        break
```

**Practical Example: Medical Diagnosis**

**Scenario**: ML model for disease detection
**Requirement**: User needs 99% accuracy for deployed predictions
**Analysis**:

| Threshold | Accuracy | Coverage |
|---|---|---|
| 0.50 | 95.2% | 98.5% |
| 0.70 | 97.1% | 94.2% |
| 0.85 | 98.5% | 87.3% |
| 0.92 | 99.1% | 76.8% |
| 0.95 | 99.5% | 68.2% |

**Decision**: Use threshold 0.92

- Achieves required 99%+ accuracy
- 76.8% of cases handled automatically
- 23.2% referred to human doctors

# 5 Ensemble Methods for Better Uncertainty

## 5.1 Why Ensembles?

**The Power of Multiple Models**

**Problem**: Single neural networks can be overconfident even when wrong!
**Solution**: Train multiple models and combine their predictions
**Key Insight**: If models disagree, we should be uncertain!
**How Ensembles Help**:

1. **Reduced variance**: Average smooths out individual model noise
2. **Better calibration**: Ensemble probabilities are more reliable
3. **Captures epistemic uncertainty**: Model disagreement = uncertainty
4. **Improved generalization**: Different models make different errors

**Analogy**: Asking 10 doctors vs 1 doctor

- If all 10 agree: High confidence
- If 5 say yes, 5 say no: Low confidence (need more investigation)

## 5.2 Building an Ensemble

**Ensemble of Neural Networks**

**Method**: Train $N$ neural networks independently, combine predictions
**Creating Diversity**: Each model should be slightly different

- Different random weight initialization
- Different random data shuffling
- (Optionally) Different subsets of training data

**Combining Predictions**: Average the probability distributions

$$P_{\text{ensemble}}(\text{class} = c) = \frac{1}{N} \sum_{i=1}^{N} P_{\text{model}_i}(\text{class} = c)$$

**Uncertainty Estimation**: Variance across models

$$\text{Uncertainty}(c) = \text{Var}\left[P_{\text{model}_i}(c)\right]_{i=1}^{N}$$

If models disagree (high variance) $\rightarrow$ High uncertainty

## 5.3 Implementation in Python

```python
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D

ENSEMBLE_SIZE = 5  # Number of models in ensemble

```

```python
# Create ensemble of CNNs
ensemble_models = []

for i in range(ENSEMBLE_SIZE):
    # Different random seed for each model
    tf.random.set_seed(i)

    model = Sequential([
        Conv2D(kernel_size=(5, 5), filters=8,
               activation='relu', padding='VALID',
               input_shape=(28, 28, 1)),
        MaxPooling2D(pool_size=(6, 6)),
        Flatten(),
        Dense(units=10, activation='softmax')
    ])

    model.compile(
        loss='sparse_categorical_crossentropy',
        optimizer='rmsprop',
        metrics=['accuracy']
    )

    ensemble_models.append(model)

# Train each model
for i, model in enumerate(ensemble_models):
    print(f"Training model {i+1}/{ENSEMBLE_SIZE}")
    model.fit(x_train, y_train, epochs=5, verbose=0)

# Evaluate individual models
for i, model in enumerate(ensemble_models):
    acc = model.evaluate(x_test, y_test, verbose=0)[1]
    print(f"Model {i+1} accuracy: {acc:.4f}")
```

## 5.4 Combining Ensemble Predictions

```python
def ensemble_predict(models, x):
    """
    Combine predictions from ensemble of models.

    Returns: averaged probability distribution
    """
    # Get predictions from all models
    predictions = np.array([model.predict(x) for model in models])
    # Shape: (num_models, num_samples, num_classes)

    # Average across models
    ensemble_probs = np.mean(predictions, axis=0)
    # Shape: (num_samples, num_classes)

    return ensemble_probs

def ensemble_uncertainty(models, x):
    """
    Compute uncertainty as variance across models.
    """
    predictions = np.array([model.predict(x) for model in models])

    # Variance for each class prediction
    variance = np.var(predictions, axis=0)

    # Total uncertainty: mean variance across classes
    uncertainty = np.mean(variance, axis=1)

```

```python
    return uncertainty

# Get ensemble predictions
ensemble_probs = ensemble_predict(ensemble_models, x_test)

# Make predictions
ensemble_predictions = np.argmax(ensemble_probs, axis=1)

# Compute ensemble accuracy
ensemble_acc = np.mean(ensemble_predictions == y_test)
print(f"Ensemble accuracy: {ensemble_acc:.4f}")

# Compute uncertainty
uncertainties = ensemble_uncertainty(ensemble_models, x_test)
```

**Ensemble Benefits**

**Typical Results** (MNIST example):

| Model | MNIST Acc. | Corrupted MNIST Acc. |
|---|---|---|
| Single CNN | 97.5% | 85.2% |
| Ensemble (5 models) | 98.1% | 87.8% |

**Key Observations**:

1. Ensemble accuracy is higher than individual models
2. Improvement is larger on out-of-distribution data
3. Ensemble uncertainty better reflects true confidence

**Trade-off**: Computational cost scales with ensemble size

- 5 models = 5x training time, 5x inference time
- Often: 3-10 models is a good balance
- Diminishing returns after 10-20 models

### 5.5 Entropy for Ensembles

```python
def ensemble_entropy(models, x):
    """
    Compute entropy of ensemble predictions.
    """
    # Get averaged probabilities
    ensemble_probs = ensemble_predict(models, x)

    # Compute entropy
    probs = np.clip(ensemble_probs, 1e-10, 1.0)
    entropy = -np.sum(probs * np.log2(probs), axis=1)

    return entropy

# Compare entropy distributions
entropy_single = compute_entropy(cnn_model.predict(x_test))
entropy_ensemble = ensemble_entropy(ensemble_models, x_test)

print(f"Single model mean entropy: {np.mean(entropy_single):.3f}")
print(f"Ensemble mean entropy: {np.mean(entropy_ensemble):.3f}")
```

# 6 Out-of-Distribution Detection

## 6.1 The OOD Problem

**Out-of-Distribution (OOD) Data**

**Definition**: Data that is significantly different from the training distribution.
**Examples**:

- Model trained on clean MNIST, tested on corrupted MNIST
- Model trained on cats/dogs, shown a car
- Model trained on English text, given French text

**The Danger**: Models often make confident predictions on OOD data!
**Goal**: Detect when input is OOD and flag for human review

**MNIST vs Corrupted MNIST**

**Setup**:

- Train model on clean MNIST images
- Test on MNIST-C (corrupted with spatters)

**Results**:

| Dataset | Accuracy | Mean Entropy |
|---|---|---|
| Clean MNIST | 97.5% | 0.15 bits |
| Corrupted MNIST | 85.2% | 0.42 bits |

**Key Observation**: Higher entropy on corrupted data!

- Model is appropriately more uncertain on OOD data
- We can use entropy threshold to detect OOD samples

## 6.2 Using Uncertainty for OOD Detection

```python
def detect_ood(model, x_in_distribution, x_test, percentile=95):
    """
    Detect out-of-distribution samples using entropy threshold.

    Args:
        model: trained model
        x_in_distribution: samples from training distribution
        x_test: samples to classify as in/out of distribution
        percentile: threshold percentile for in-distribution entropy

    Returns:
        is_ood: boolean array indicating OOD samples
    """
    # Compute entropy on known in-distribution data
    probs_id = model.predict(x_in_distribution)
    entropy_id = compute_entropy(probs_id)

    # Set threshold at specified percentile
    threshold = np.percentile(entropy_id, percentile)

```

```
    # Compute entropy on test data
    probs_test = model.predict(x_test)
    entropy_test = compute_entropy(probs_test)

    # Flag samples with entropy above threshold as OOD
    is_ood = entropy_test > threshold

    return is_ood, threshold

# Detect OOD samples in corrupted MNIST
is_ood, threshold = detect_ood(
    cnn_model,
    x_test,          # In-distribution
    x_c_test,        # Potentially OOD
    percentile=95
)

print(f"Entropy threshold: {threshold:.3f}")
print(f"Samples flagged as OOD: {np.sum(is_ood)} / {len(is_ood)}")
print(f"OOD detection rate: {100 * np.mean(is_ood):.1f}%")
```

# 7 Complete Implementation Example

## 7.1 Full Pipeline: Training to Uncertainty-Aware Prediction

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D

# ============================================================
# 1. DATA LOADING
# ============================================================

def load_and_preprocess_data():
    """Load MNIST and normalize."""
    (x_train, y_train), (x_test, y_test) = \
        tf.keras.datasets.mnist.load_data()

    # Normalize and add channel dimension
    x_train = x_train.astype('float32') / 255.0
    x_test = x_test.astype('float32') / 255.0
    x_train = x_train[..., np.newaxis]
    x_test = x_test[..., np.newaxis]

    return (x_train, y_train), (x_test, y_test)

(x_train, y_train), (x_test, y_test) = load_and_preprocess_data()

# ============================================================
# 2. BUILD ENSEMBLE
# ============================================================

ENSEMBLE_SIZE = 5

def build_model():
    """Build a single CNN model."""
    return Sequential([
        Conv2D(8, (5, 5), activation='relu', input_shape=(28, 28, 1)),
        MaxPooling2D((6, 6)),
        Flatten(),
        Dense(10, activation='softmax')
    ])
```

```python

# Create and train ensemble
ensemble = []
for i in range(ENSEMBLE_SIZE):
    tf.random.set_seed(i)
    model = build_model()
    model.compile(optimizer='rmsprop',
                  loss='sparse_categorical_crossentropy',
                  metrics=['accuracy'])
    model.fit(x_train, y_train, epochs=5, verbose=0)
    ensemble.append(model)
    print(f"Trained model {i+1}/{ENSEMBLE_SIZE}")

# ============================================================
# 3. UNCERTAINTY-AWARE PREDICTION
# ============================================================

def predict_with_uncertainty(ensemble, x):
    """
    Make predictions with uncertainty estimates.

    Returns:
        predictions: predicted class labels
        confidence: maximum probability for each prediction
        entropy: entropy of probability distribution
        ensemble_std: standard deviation across ensemble
    """
    # Get all model predictions
    all_probs = np.array([m.predict(x, verbose=0) for m in ensemble])

    # Average probabilities
    avg_probs = np.mean(all_probs, axis=0)

    # Predictions and confidence
    predictions = np.argmax(avg_probs, axis=1)
    confidence = np.max(avg_probs, axis=1)

    # Entropy
    probs_clipped = np.clip(avg_probs, 1e-10, 1.0)
    entropy = -np.sum(probs_clipped * np.log2(probs_clipped), axis=1)

    # Ensemble disagreement (std across models)
    ensemble_std = np.mean(np.std(all_probs, axis=0), axis=1)

    return predictions, confidence, entropy, ensemble_std

# Make predictions
preds, conf, ent, std = predict_with_uncertainty(ensemble, x_test)

# ============================================================
# 4. SELECTIVE PREDICTION
# ============================================================

def selective_predict(predictions, confidence, threshold):
    """Make predictions only when confident enough."""
    mask = confidence >= threshold
    return predictions[mask], mask

# Try different thresholds
for threshold in [0.7, 0.8, 0.9, 0.95]:
    selected_preds, mask = selective_predict(preds, conf, threshold)
    coverage = 100 * np.mean(mask)
    accuracy = 100 * np.mean(selected_preds == y_test[mask])
    print(f"Threshold {threshold}: {accuracy:.1f}% acc, {coverage:.1f}% cov")
```

## 8 Key Takeaways

Uncertainty Estimation: Summary

### 1. Why Uncertainty Matters

**Core Problem**: Traditional models give predictions without confidence
**Solution**: Probabilistic models that output distributions
**Benefits**:

- Know when model might be wrong
- Enable selective prediction
- Detect out-of-distribution data
- Safer deployment in critical applications

### 2. Softmax for Probabilistic Output

**Formula**:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

**Properties**:

- Outputs sum to 1 (valid probability distribution)
- Higher logits $\rightarrow$ higher probability
- Interpretable as class probabilities

### 3. Entropy Measures Uncertainty

**Formula**:

$$H = -\sum_{j=1}^{K} p_j \log_2(p_j)$$

**Interpretation**:

- Low entropy ($\approx 0$): Model is confident
- High entropy ($\approx \log_2 K$): Model is uncertain
- $2^H$ = effective number of classes under consideration

**Key Finding**: Models have higher entropy on incorrect predictions!

### 4. Selective Prediction

**Concept**: Only predict when confident, abstain otherwise
**Metrics**:

- **Coverage**: Fraction of samples where prediction is made
- **Selective Accuracy**: Accuracy on samples where prediction is made

**Trade-off**: Higher threshold $\rightarrow$ Lower coverage, Higher accuracy

## 5. Ensemble Methods

**Idea**: Combine multiple models for better uncertainty
**Implementation**:

$$P_{\text{ensemble}}(c) = \frac{1}{N} \sum_{i=1}^{N} P_{\text{model}_i}(c)$$

**Benefits**:

- Better calibrated probabilities
- Model disagreement indicates uncertainty
- Improved accuracy and robustness

**Cost**: $N\times$ training and inference time

## 6. Key Formulas to Remember

**Softmax**:

$$P(y = i \mid \vec{x}) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

**Entropy (bits)**:

$$H = -\sum_i p_i \log_2(p_i)$$

**Coverage**:

$$\text{Coverage} = \frac{\#\text{predictions made}}{\#\text{total samples}}$$

**Selective Accuracy**:

$$\text{Selective Acc} = \frac{\#\text{correct predictions made}}{\#\text{predictions made}}$$

## 7. Practical Guidelines

1. **Always use softmax** for classification output layers
2. **Monitor entropy** to understand model uncertainty
3. **Use selective prediction** for high-stakes applications
4. **Consider ensembles** when uncertainty estimation is critical
5. **Test on OOD data** to verify uncertainty calibration
6. **Balance coverage vs accuracy** based on application needs

## 8. When to Use What

| Situation | Approach |
|---|---|
| Basic probability output | Softmax activation |
| Measure prediction uncertainty | Compute entropy |
| Need very high accuracy | Selective prediction |
| Critical application | Ensemble methods |
| Detect unusual inputs | OOD detection with entropy |

**9. Common Pitfalls**

- **Don't trust raw softmax blindly**: Models can be overconfident
- **Threshold selection matters**: Validate on held-out data
- **OOD data is tricky**: Models may still be confident on novel inputs
- **Ensemble size**: Diminishing returns after 5-10 models
- **Computational cost**: Ensembles multiply inference time